SPAM / HAM SMS CLASSIFIER

A Machine Learning Project Report

Submitted by: Anivartak

Date: October 22, 2026

Abstract

Almost everyone with a phone gets spam messages. In this project I built a small machine learning model that reads an SMS and decides whether it is spam or ham (a normal message). The text is cleaned first and then turned into numbers using TF-IDF. A Multinomial Naive Bayes classifier then makes the prediction.

I trained the model on the SMS Spam Collection dataset and tested it on 1,035 messages it had never seen. It reached an accuracy of 98.94%, with a precision of 97.62% and a recall of 93.89% for the spam class. The whole project was run in Google Colab.

1. Introduction

Spam messages are annoying, and some of them are dangerous because they try to trick people into clicking a link or sharing bank details. A spam filter reads each incoming message and decides whether it should reach the inbox. This is a classification problem with two classes, spam and ham.

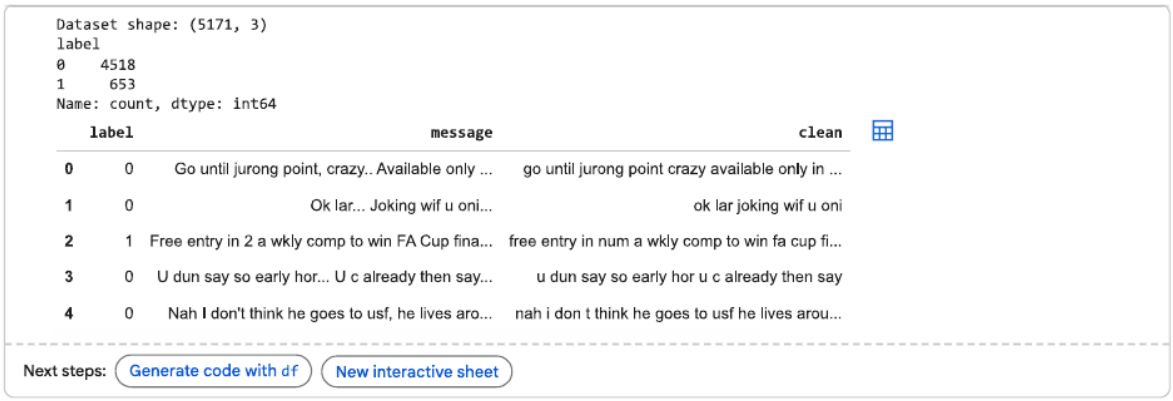
One thing that makes the problem harder is that real messages are not balanced. Most messages are ham, so a model could get a high accuracy just by calling everything ham. For that reason, precision and recall for the spam class were checked along with accuracy.

The aim of this project was to build a complete working classifier: clean the text, train a model, test it properly, and keep the code and results ready to put on GitHub.

2. Dataset

The dataset is the SMS Spam Collection from the UCI Machine Learning Repository, which is also available on Kaggle. It is a CSV file called `spam.csv` with two columns: `v1` holds the label (ham or spam) and `v2` holds the message text.

The file has 5,574 messages, of which 4,827 are ham and 747 are spam. After removing duplicate rows, 5,171 messages were left, 4,518 ham and 653 spam, so spam is about 12.6% of the data. The labels were changed to numbers, 0 for ham and 1 for spam. Because spam is so much rarer than ham, the train and test split was stratified so that both parts keep the same proportion. Fig. 1 shows the output of the loading step, with the shape of the data, the class counts and the first five messages next to their cleaned versions.



```
Dataset shape: (5171, 3)
label
0    4518
1     653
Name: count, dtype: int64
```

	label	message	clean
0	0	Go until jurong point, crazy.. Available only ...	go until jurong point crazy available only in ...
1	0	Ok lar... Joking wif u oni...	ok lar joking wif u oni
2	1	Free entry in 2 a wkly comp to win FA Cup fina...	free entry in num a wkly comp to win fa cup fi...
3	0	U dun say so early hor... U c already then say...	u dun say so early hor u c already then say
4	0	Nah I don't think he goes to usf, he lives aro...	nah i don t think he goes to usf he lives arou...

Next steps: Generate code with df | New interactive sheet

Fig. 1. Output of the data loading step in Colab.

3. Tools Used

Tool	Used for
Python 3	Writing the program
pandas	Loading and cleaning the dataset
scikit-learn	TF-IDF, Naive Bayes model, pipeline and evaluation metrics
matplotlib	Drawing the confusion matrix
joblib	Saving the trained model
Google Colab	Running the notebook online
Git and GitHub	Keeping the project and sharing it

4. Methodology

4.1 Cleaning the text

Raw SMS text is messy, so every message goes through a function called `clean_text` before it reaches the model. It converts the text to lowercase, replaces web links with the word `url`, replaces numbers with the word `num`, removes punctuation and special characters, and removes extra spaces. For example, "Ok lar... Joking wif u oni..." becomes "ok lar joking wif u oni".

Links and numbers were replaced instead of deleted because spam often has phone numbers, prices and links in it. The model can learn that having a number or link is a useful sign, without remembering the exact number.

4.2 Splitting the data

The cleaned data was split into 80% for training and 20% for testing, which gave 4,136 training messages and 1,035 test messages. The split uses `random_state=42` so the results can be repeated, and `stratify` so that the spam ratio is the same in both parts.

4.3 TF-IDF

A model cannot read words directly, so the messages are converted to numbers with TF-IDF (Term Frequency and Inverse Document Frequency). A word gets a higher weight when it appears often in one message but not in most other messages. Words like "free", "claim" and "prize" get noticed this way, while very common words get a low weight. The vectorizer removes English stop words and uses both single words and pairs of words (`ngram_range=(1, 2)`), so phrases like "free entry" and "call now" are also captured.

4.4 Multinomial Naive Bayes

The classifier is Multinomial Naive Bayes. It uses Bayes' theorem to find the probability of each class given the words in the message, and it assumes the words are independent of each other. This assumption is not really true for language, but the model still works well for text, it is fast, and it does not need a lot of data. The smoothing value was set to `alpha=0.1` so that a word never seen in one class does not make the probability zero.

The TF-IDF step and the classifier were put together in one scikit-learn `Pipeline`. This way the vectorizer is fitted only on the training data, and a new message can go through the same steps with one call.

5. Implementation

The project has these files: `spam_classifier_colab.ipynb` (the Colab notebook), `spam_classifier.py` (the same program for running on a computer), `spam.csv` (the dataset), `requirements.txt` (the libraries needed) and `README.md`. The screenshots below are from the Colab notebook, which is run from top to bottom.

5.1 Importing libraries

The first cell imports the libraries for text cleaning, data handling, the model and the evaluation.

```
import re
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import (ConfusionMatrixDisplay, accuracy_score,
                             classification_report, f1_score,
                             precision_score, recall_score)
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline
```

Fig. 2. Importing the libraries.

5.2 Uploading the dataset

In Colab the file is uploaded with `files.upload()`. The output confirms that `spam.csv` (480,378 bytes) was uploaded.

```
from google.colab import files
uploaded = files.upload()
```

```
Choose Files spam.csv
spam.csv(text/csv) - 480378 bytes, last modified: 10/22/2026 - 100% done
Saving spam.csv to spam.csv
```

Fig. 3. Uploading spam.csv.

5.3 Loading and cleaning the data

This cell defines `clean_text`, reads the file, removes duplicates and empty rows, converts the labels to 0 and 1, and creates a new column with the cleaned text. Its output is shown in Fig. 1.

```
def clean_text(text):
    text = text.lower()
    text = re.sub(r"http\S+|www\S+", " url ", text)
    text = re.sub(r"\d+", " num ", text)
    text = re.sub(r"[^a-z\s]", " ", text)
    return re.sub(r"\s+", " ", text).strip()

df = pd.read_csv("spam.csv", encoding="latin-1")[["v1", "v2"]]
df.columns = ["label", "message"]
df = df.drop_duplicates().dropna()
df["label"] = df["label"].map({"ham": 0, "spam": 1})
df["clean"] = df["message"].apply(clean_text)

print("Dataset shape:", df.shape)
print(df["label"].value_counts())
df.head()
```

Fig. 4. Loading and cleaning the data.

5.4 Training the model

Here the data is split and the pipeline is trained. The line "Training complete" is printed when the model has been fitted.

```
X_train, X_test, y_train, y_test = train_test_split(
    df["clean"], df["label"], test_size=0.2, random_state=42, stratify=df["label"])

model = Pipeline([
    ("tfidf", TfidfVectorizer(stop_words="english", ngram_range=(1, 2))),
    ("nb", MultinomialNB(alpha=0.1)),
])
model.fit(X_train, y_train)
print("Training complete")
```

```
Training complete
```

Fig. 5. Splitting the data and training the model.

5.5 Evaluating the model

This cell predicts the test messages, prints the metrics and the classification report, and draws the confusion matrix. The output is in Section 6.

```
pred = model.predict(X_test)
print(f"Accuracy : {accuracy_score(y_test, pred):.4f}")
print(f"Precision: {precision_score(y_test, pred):.4f}")
print(f"Recall   : {recall_score(y_test, pred):.4f}")
print(f"F1-score : {f1_score(y_test, pred):.4f}\n")
print(classification_report(y_test, pred, target_names=["ham", "spam"]))

ConfusionMatrixDisplay.from_predictions(y_test, pred, display_labels=["ham", "spam"], cmap="Blues")
plt.title("Confusion Matrix")
plt.show()
```

Fig. 6. Code for evaluating the model.

5.6 Predicting new messages

The `predict` function cleans a message in the same way as the training data and gives back the class and how confident the model is. The results for three test messages are in Section 6.4.

```
def predict(message):
    x = [clean_text(message)]
    label = model.predict(x)[0]
    prob = model.predict_proba(x)[0][label]
    return f"{'SPAM' if label else 'HAM'} ({prob:.1%} confident)"

tests = [
    "Congratulations! You won a free iPhone. Click http://win.com now",
    "Hey, are we still meeting for lunch tomorrow?",
    "URGENT! Your account is blocked. Call 09061234567 to claim",
]
for t in tests:
    print(f"{t[:55]:<57} -> {predict(t)}")
```

5.7 Running on a computer

The same program can be run locally. After installing the libraries, the first command trains the model and the second one checks a single message.

```
pip install -r requirements.txt
python spam_classifier.py
python spam_classifier.py --predict "You won a free prize!"
```

6. Results

6.1 Metrics

The model was tested on 1,035 messages that were not used in training. The scores for the spam class are given below.

Metric	Score
Accuracy	0.9894
Precision	0.9762
Recall	0.9389
F1-score	0.9572

Fig. 7 is the evaluation output from Colab. It also has the classification report, with the scores for ham and spam separately.

```
Accuracy : 0.9894
Precision: 0.9762
Recall   : 0.9389
F1-score : 0.9572

              precision    recall  f1-score   support

         ham       0.99      1.00      0.99       904
        spam       0.98      0.94      0.96       131

    accuracy                           0.99      1035
   macro avg       0.98      0.97      0.98      1035
weighted avg       0.99      0.99      0.99      1035
```

Fig. 7. Evaluation output in Colab.

6.2 Confusion matrix

The confusion matrix in Fig. 8 shows how many messages of each class were predicted correctly and wrongly. The same numbers are in the table below it.

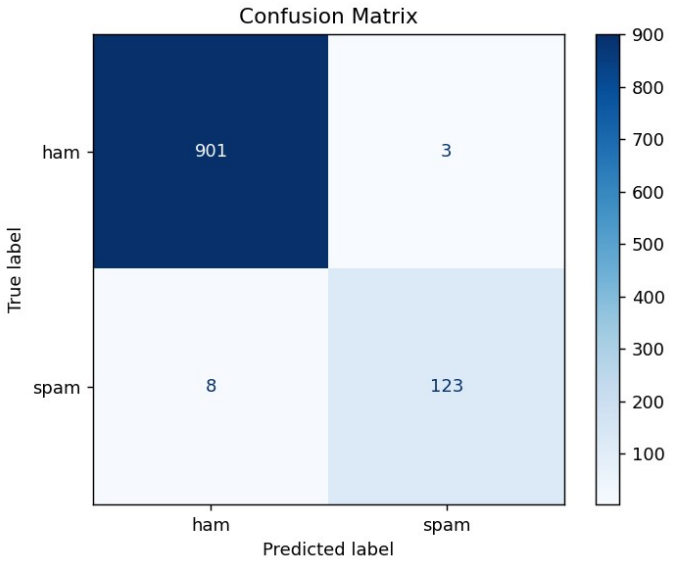


Fig. 8. Confusion matrix for the test set.

	Predicted ham	Predicted spam
Actual ham	901	3
Actual spam	8	123

6.3 Discussion

Out of 904 ham messages in the test set, only 3 were wrongly marked as spam. This is the mistake that matters most for a spam filter, because a normal or important message could end up in the spam folder. The precision of 97.6% shows that the model makes this mistake very rarely.

Out of 131 spam messages, 8 were missed, which gives a recall of 93.9%. These messages would simply reach the inbox, which is less harmful than blocking a real message. The accuracy of 98.9% looks very good, but a model that called every message ham would already get about 87% on this data, so precision and recall are the numbers that really tell how good the model is.

6.4 Sample predictions

Message	Prediction
"Congratulations! You won a free iPhone. Click http://win.com now"	SPAM (93.4%)
"Hey, are we still meeting for lunch tomorrow?"	HAM (99.9%)
"URGENT! Your account is blocked. Call 09061234567 to claim"	SPAM (98.5%)

The model picked up the usual spam signs such as prizes, urgent words, links and phone numbers, and it treated the normal message as ham.

7. Limitations

The dataset is small and quite old, so newer kinds of spam, like UPI or bank phishing and fake delivery messages, may not be caught well. It also contains only English, so messages in Hindi or Hinglish will not work properly. TF-IDF looks only at which words are present and not at their meaning or order, so a cleverly written spam message can get through. Finally, the test set has only 131 spam messages, so the scores could change a little with a different split.

8. Future Work

The model can be improved by trying other algorithms such as Logistic Regression, SVM and Random Forest, and by tuning the settings with GridSearchCV and cross-validation. Extra features like message length and the number of links could be added. A bigger and newer dataset, including Hindi and Hinglish messages, would make it more useful in real life. The

model could also be turned into a small web app or an API, for example with Flask or a Spring Boot service.

9. Conclusion

In this project I built an SMS spam classifier using text cleaning, TF-IDF and Multinomial Naive Bayes. It reached 98.94% accuracy on the test data, with 97.62% precision and 93.89% recall for spam, so it rarely blocks a real message and still catches most of the spam. Working on it helped me understand the full process of a machine learning project, from preparing the data to testing the results.

References

[1] T. A. Almeida, J. M. Gómez Hidalgo and A. Yamakami, "Contributions to the study of SMS spam filtering: new collection and results," ACM Symposium on Document Engineering, 2011. (SMS Spam Collection, UCI Machine Learning Repository)

[2] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, 2011.

[3] scikit-learn documentation, scikit-learn.org.

[4] SMS Spam Collection Dataset, Kaggle, kaggle.com/datasets/uciml/sms-spam-collection-dataset.